

[Skip to content](#)



ChatGPT
▼



You said:

what happens in a transformer encoder-decoder model? how are embedding layers and head weights initialized?



ChatGPT said:

Great question — before we dive in, let me check in with you so I can pitch this at the right level:

What's your current background with transformers?

For example, have you already studied:

- self-attention,
- encoder–decoder architectures like in T5 or original Transformer,
- or weight initialization schemes like Xavier/Glorot?

Just a quick sense so I know whether to start from the foundations or build on more advanced knowledge.



You said:

i've already studied self-attention



ChatGPT said:

Great — since you already know self-attention, let's build from there. I'll walk you through the encoder–decoder flow, but I'll guide you with small steps so you can connect it to what you know.

Step 1 — What *extra* attention does a decoder need?

You already know that in an encoder, every token attends to every other token.

A decoder has **two** kinds of attention. One of them is self-attention (but masked). The other one is new.

Quick question to anchor you:

👉 Besides masked self-attention, what other attention operation do you think the decoder must perform in order to use information from the encoder?

(Just take a guess — we'll build from there.)



You said:

attend to the result?



ChatGPT said:

Exactly — you're pointing in the right direction.

The decoder needs to *look at* (attend to) the encoder's output. Let's sharpen that a bit.

✓ What the decoder attends to

In addition to masked self-attention, each decoder layer has:

◆ Cross-attention:

The decoder queries the **encoder's output representations** (often called the "encoder memory").

So the flow becomes:

1. Decoder token → masked self-attention (looks only left)
2. Same token → **cross-attention** (looks at encoder outputs)
3. Then a feed-forward layer

To make sure this stuck, here's a quick check-in:

👉 **If the decoder didn't have cross-attention, what important ability would it lose?**
(Think: what does a decoder need from the encoder?)



You said:

the format of the original representation



ChatGPT said:

Nice intuition — you're circling the idea. Let me help you tighten it:

The decoder would lose access to **the meaning of the input sequence** — the entire encoded representation of it. Without cross-attention, the decoder would just autocomplete its own output without knowing what it's translating or responding to.

But you're on the right track: cross-attention is how the decoder connects its generation to the encoder's understanding.

Now that you've got the *function* of the encoder–decoder setup, let's shift to your second question:
How are embedding layers and output head weights initialized?

I'll guide you through this step by step.

Step 2 — Before answering: what do you already know?

There are a few common initialization schemes you might have seen:

- Xavier/Glorot initialization
- Kaiming/He initialization
- Normal distributions with small std
- Weight tying between embeddings and output heads

To make sure I build on what you know:

👉 **Which of these (if any) have you encountered before in the context of neural nets?**



You said:

all of them except for weight tying between embeddings and output heads



ChatGPT said:

Perfect — then we can build smoothly from what you know.
Let's walk through initialization in encoder–decoder transformers, but I'll guide you with small steps.

Step 3 — How embeddings & output heads are initialized

Let's build this piece by piece so you can form the full picture yourself.

First idea:

In many encoder–decoder models (like T5, BART, original Transformer), the **token embedding matrix** and the **final linear layer that maps decoder hidden states to vocabulary logits** often share the *same* weight matrix — this is called **weight tying**.

To make sure the idea lands:

👉 **Why might it be beneficial to use the *same* matrix for both:**

- embedding tokens → vectors
- projecting hidden states → vocab logits

What advantage do you think this gives the model?
(Just take a guess — there's no wrong answer, we'll refine it.)



You said:

reduces parameter count, creates redundancies



ChatGPT said:

Exactly — **reducing parameter count** is a big one. And your point about “creating redundancies” is close to another key idea: it forces *consistency* between how tokens are represented going in and how they’re predicted going out.

Let me sharpen that with you:

Why weight tying helps

- ✓ **Fewer parameters** → less overfitting, faster training
- ✓ **Shared geometry** → the space used to embed tokens is the *same* space the model uses to score them during generation
- ✓ **Easier optimization** → the softmax head doesn’t have to learn the mapping from scratch

Good.

Step 4 — Now: how are these weights actually initialized?

Let’s build from what you already know.

Most transformer implementations do:

Token Embeddings

Initialized using **normal(0, $1/\sqrt{d_{\text{model}}}$)** or **Xavier uniform**, depending on framework.

Positional Embeddings

Same style, but often **normal(0, 0.02)** (GPT, BERT) or **sinusoidal with no learned parameters** (original Transformer).

Attention Weights

All projection matrices (Q, K, V, O) → usually **Xavier uniform or normal**, because they are linear layers.

Feedforward Network

Same as above — **Xavier** for the fully-connected layers.

Output Head

If weight tying is used → it *reuses* the token embedding matrix (so it’s initialized exactly the same way).

If not tied → it uses **Xavier** like any linear layer.

I want to make sure all of this fits together in your head, so let’s do a quick check.

👉 **Given what we discussed, what do you think happens to the output head weights at initialization when weight tying *is* used?**
(Just explain in your own words — 1 sentence is enough.)



You said:

what should the dimensions of the embedding matrix for tokens and positions be?



ChatGPT said:

Good — let’s figure this out **together**, step-by-step, instead of me just giving the shapes.

Since you’ve studied self-attention, you already know every token needs to be represented as a **vector of size d_{model}** so it can flow through attention blocks.

So let’s reason it out:

🧠 Guiding question 1

If you have a vocabulary with V tokens, and each token needs a vector of size d_{model} ,

👉 **what dimensions do you think the token embedding matrix must have?**

(Try expressing it as something like $____ \times d_{\text{model}}$.)

After that, we’ll work out positional embeddings the same way.



You said:

$V \times d_{\text{model}}$



ChatGPT said:

Exactly — perfect.

- **Token embedding matrix:** $V \times d_{\text{model}}$

Each of the V tokens maps to a row vector of size d_{model} .

Good. Now let's reason about positional embeddings.



Guiding question 2 — Positional embedding shape

The model needs one learned vector per **position index** (0, 1, 2, ..., max sequence length - 1), each of size d_{model} .

So let's think this through:

If the **maximum sequence length** is L , and each position also needs a vector of size d_{model} ...

👉 **What would the dimensions of the positional embedding matrix be?**
(Use L and d_{model} .)



You said:

why are token embeddings initialized using $\text{normal}(0, 1/\sqrt{d_{\text{model}}})$ and not $\text{normal}(0, 1)$?



ChatGPT said:

Great question — and this is exactly the kind of thing that clicks best if you reason it out yourself. Let's guide you there step by step.



Step 1 — Start from something you already know

You've seen Xavier/Glorot initialization, right?

Those methods choose a variance that depends on the **fan-in** / **fan-out** of the layer so that:

- activations don't explode
- activations don't vanish
- gradients stay stable

Embedding layers are *also* linear maps (just looked up by index instead of multiplied), so the same logic applies.

Let's reason from first principles.



Step 2 — Guiding question

If each embedding vector had entries drawn from **normal(0, 1)**:

Each token vector is size d_{model} .

The *expected squared norm* of such a vector would be:

$$E[\|x\|^2] = d_{\text{model}} \cdot \text{Var}(x_i)$$

$$E[\|x\|^2] = d_{\text{model}} \cdot \text{Var}(x_i)$$

If $\text{Var} = 1$, then:

$$E[\|x\|^2] = d_{\text{model}}$$

$$E[\|x\|^2] = d_{\text{model}}$$

👉 **What happens to the expected norm of the embedding vector as d_{model} gets large?**
(Just describe it qualitatively — explodes? shrinks? stays the same?)



You said:

then the expected norm will blow up -- we want it to stay at 1



ChatGPT said:

Exactly — perfect intuition.

So now you can probably see where this is heading:

If we want the **expected norm of each embedding vector to stay around 1**, then each component needs variance:

$$\text{Var} = \frac{1}{d_{\text{model}}}$$

`Var = dmodel1`

And that's exactly why frameworks use:

$$N\left(0, \frac{1}{\sqrt{d_{\text{model}}}}\right)$$

`N(0, dmodel`

`1)`

because variance = (std)^2 = 1 / d_model.

Step 3 — Tie it back to Xavier initialization

This is basically the same reasoning Xavier uses:

- You want the output of a linear operation to have variance that doesn't depend on the layer width.
- Embeddings behave like the input layer of a linear network.
- So you use variance scaled by fan-in (d_model).

✓ Quick comprehension check

Just to reinforce the idea:

 If we used `std = 1` instead of `1/√d_model`, what would happen as the embedding goes through several layers of attention and feedforward blocks?

(Think in terms of activations blowing up or shrinking.)



You said:

why are position embeddings initialized using `normal(0, 0.02)`?



ChatGPT said:

Ah, good — now we're on positional embeddings. Let's reason through this carefully.

Step 1 — What's different about positional embeddings?

- Token embeddings correspond to **discrete vocabulary items** — many tokens, each has a vector.
- Positional embeddings correspond to **positions in a sequence** — usually a much smaller number (L_max), and the positions are sequential and correlated.
- Positional embeddings **don't have any input-dependent variance**; they just inject information about position.

So the initialization doesn't need to scale with d_model in the same way as token embeddings.

Step 2 — Why such a small standard deviation?

Frameworks (like GPT, BERT) often use `N(0, 0.02)` for learned position embeddings. The reasons are:

1. **Keep initial values small** → prevent early layers from being dominated by positional info.

2. **Avoid exploding activations** → small initial positions plus attention layers ensure the network can learn relative positions without extreme values.
3. **Stable training** → since positions are shared across sequences, having a small variance makes optimization easier.

Step 3 — Quick intuition check

Think of it like this: token embeddings carry the “semantic weight” (what the word means), while positional embeddings just give a **hint about order**. They don’t need to have large magnitudes at first; the model will learn how much to amplify them.

If you want, I can also draw a **tiny diagram comparing token vs positional embedding initialization**, which makes this super clear.

Do you want me to do that?



You said:

using what you, can you give me some relevant study problems?



ChatGPT said:

Absolutely — and I’ll follow the study-mode rules: guiding, building on what you know, and asking one question at a time.

Here are **5 short, targeted practice problems** about transformer initialization and encoder–decoder mechanics. Each one is designed so you can reason it out — not memorize.

I’ll give you **Problem 1** now. After you answer, I’ll give feedback and move you to Problem 2.

Problem 1 — Embedding variance reasoning

Suppose a model has $d_{\text{model}} = 1024$.

A token embedding vector is initialized with

$$x_i \sim N(0, 1/\sqrt{d_{\text{model}}}).$$

$x_i \sim N(0, 1/\sqrt{d_{\text{model}}})$.

👉 **What is the expected squared norm of the embedding vector?**

(You don’t have to calculate numerically — just express it in terms of d_{model} and the variance.)

Take your time — what do you think?



You said:

1



ChatGPT said:

Nice — that’s correct **if** you applied the reasoning we developed:

- Each component has variance $1/d_{\text{model}}$
- There are d_{model} components
- Expected squared norm = $d_{\text{model}} \cdot (1/d_{\text{model}}) = 1$

Good — you’re thinking exactly the right way.

Ready for the next one?

Problem 2 — Cross-attention logic

In an encoder–decoder transformer, the decoder has two attention sublayers:

1. Masked self-attention
2. Cross-attention (queries from decoder, keys/values from encoder)

👉 If the cross-attention layer were removed, what specific capability would the model lose during generation?

(Answer in one sentence.)



You said:

spatial understanding



ChatGPT said:

You're on the right wavelength, but let's refine it a bit.

"Spatial understanding" is close metaphorically, but let's make it more precise:

Cross-attention is the *only path* the decoder has to access **the encoded representation of the input sequence**.

So without it, the decoder would essentially become a language model that only autocompletes its own output and **cannot condition on the input**.

Try to restate that idea in your own words:

👉 What does the decoder stop being able to "look at" if cross-attention is removed?



You said:

it wouldn't have a reference to the original distribution



ChatGPT said:

Good — that's pointing in the right direction. Let's make it sharper together.

When you say "*original distribution*", I think you're trying to describe the representation the encoder produces. That representation **is** a distribution of information about the input — its meaning, structure, context.

Let me guide you a bit:

The encoder takes the input sequence and produces a set of hidden vectors (one per token). These vectors **are exactly what the decoder "looks at" during cross-attention**.

So try this:

👉 What are those encoder hidden vectors representing?

(Think: what information do they carry about the input?)



You said:

they carry latent information about the input (how curvy are the digits, etc.)



ChatGPT said:

Yes — exactly. They carry the **latent representation**: all the compressed, contextualized meaning of the input.

Let's wrap this problem up:

If cross-attention were removed, the decoder would lose access to *that* information — so it couldn't condition its output on the input at all.

Great. Let's keep building.

Problem 3 — Positional embedding initialization

Learned positional embeddings are often initialized from a small distribution like

$$N(0, 0.02).$$

$N(0, 0.02)$.

👉 Why do you think positional embeddings are initialized much smaller than token embeddings?

(Just give your best reasoning — 1–2 sentences.)



You said:

because only small positions are relevant for attention



ChatGPT said:

You're circling the right idea — let me help you refine it with a nudge rather than giving the full answer outright.

When you say “*only small positions are relevant for attention*,” you're hinting at the fact that positional information shouldn't overpower token meaning.

Here's a guiding prompt:

👉 If positional vectors started very large, what might happen to the *sum* of (token embedding + positional embedding) that gets fed into the encoder?

Would the model “see” mostly the token meaning, or mostly the position?



You said:

token meaning is more important than position, we want the model to see mostly the meaning



ChatGPT said:

Exactly — that's the key idea.

You've now articulated the reasoning behind the small variance:

- **Token meaning should dominate early training.**
- **Position is a gentle hint**, not the main signal.
- So positional embeddings start tiny so they don't drown out token embeddings.

Nice work.

Ready for the next one?

Problem 4 — Weight tying

Suppose we use weight tying between the token embedding matrix E (shape $V \times d_{\text{model}}$) and the output projection matrix W_{out} used before the softmax.

👉 If they are tied, what is the shape of W_{out} ?
(Just give the dimension — no explanation needed.)

